Kaggle 모델 추천

08/31/2026 23:08:49
https://chatgpt.com/c/6a965bc6-0924-83e8-8b93-e292465c1517

I’m working on a data science assignment using ChatGPT with the latest…

For this assignment, I’d pick something that is popular enough to look…

Sounds good. Then, let's get started with House Prices dataset. Please bre…

Absolutely. We’ll structure the entire project around CRISP-DM (Cross-…

train.csv Here is the train.csv

Great — we have the dataset. I’ll start with CRISP-DM Phase 1 and the first…

Can we continue in the Chat mode?

Yes — we can stay entirely in Chat mode. I’ll work directly from your…

Don't stop until you have the final result of this.

I’ll carry this all the way through CRISP-DM: EDA, preparation, multiple…

I’m working on a data science assignment using ChatGPT with the latest thinking model 5. I need to select a popular Kaggle dataset, perform an end-to-end data science workflow, understand and explain each step. And later I need to make a report and the quality should be as good as to turn the results into a Medium article. First of all, can you recommend me a good model for this assignment?

2026. 8. 31. 오후 9:59:49

For this assignment, I’d pick something that is **popular enough to look credible, but rich enough to produce a strong story and visuals for Medium**. I’ll recommend a dataset + model combination rather than just a model, because the pairing matters a lot here.

Thinking

웹 검색 중

kaggle.com

모델과 데이터 고르기

이 과제에는 Kaggle의 House Prices 데이터를 추천해요. 작은 표 형식 데이터라 결측치 처리와 특성 공학을 보여주기 좋고, 기본 선형회귀와 Random Forest를 비교한 뒤 Gradient Boosting이나 XGBoost를 최종 모델로 선택하면 분석 흐름이 탄탄해져요.

모델과 데이터를 선택함

웹사이트 1개 검색함

✓ 11s 동안 처리함

For this assignment, I’d recommend building the project around **Kaggle’s House Prices: Advanced Regression Techniques** dataset and using **Gradient Boosting as your main machine-learning model**.

Kaggle specifically positions House Prices as a dataset for students who already know some Python/ML basics and want to practice **feature engineering, random forests, and gradient boosting**. It has **79 explanatory variables**, which gives us plenty to analyze without the dataset becoming enormous. (Kaggle)

Kaggle — House Prices: Advanced Regression Techniques

Why I think this is a great choice for your assignment

The target is very easy to understand:

> ***Given information about a house → predict its sale price.***

That lets us demonstrate almost the entire data-science workflow naturally:

1. **Understand the problem and dataset**
2. **Load and inspect the data**
3. **Exploratory Data Analysis**
    - Sale-price distribution
    - Correlations
    - Neighborhood vs. price
    - Living area vs. price
    - Overall quality vs. price
4. **Data cleaning**
    - Missing values
    - Numerical vs. categorical variables
    - Outliers
5. **Feature engineering**
6. **Train/test split**
7. **Baseline model - Linear Regression**

8. **Random Forest**
9. **Gradient Boosting**
10. **Compare model performance**
11. **Interpret the winning model**
    - feature importance
    - optionally SHAP
12. **Visualize predictions and errors**
13. **Explain conclusions in normal English**

That gives you much more material for a Medium article than simply saying, "I trained a neural network and got 86% accuracy."

The model strategy I recommend

I actually **wouldn't start with deep learning**.

We can make the article more educational by showing progressively better models:

Model 1 - Linear Regression

This becomes our baseline.

We can explain:

> *"What happens if we assume house price is approximately a weighted combination of its characteristics?"*

Then:

Model 2 - Random Forest Regressor

Now we introduce nonlinear relationships and decision trees.

Then:

Model 3 - Gradient Boosting Regressor

This would probably become our main model.

Gradient boosting is particularly appropriate because Kaggle itself recommends practicing gradient boosting for this competition. (Kaggle)

And optionally:

Model 4 - Neural Network / MLP

Since your professor mentioned that deep learning is ideal, we could add a small neural network experiment near the end.

That gives us an interesting conclusion:

> ***Does deep learning actually outperform traditional machine learning on structured tabular data?***

That's a much better article story than forcing deep learning to be the centerpiece.

Why I prefer this over Titanic

Titanic is probably the most famous beginner Kaggle dataset. Kaggle itself describes it as its introductory ML competition. (Kaggle)

But that's also its weakness for **your assignment**.

Thousands upon thousands of tutorials already look like:

> *Age → Sex → Passenger Class → Random Forest → Predict survival*

It can feel like another beginner tutorial.

House Prices lets us produce much richer charts and explanations like:

"What Really Determines the Price of a House? A Complete Machine Learning Analysis with ChatGPT"

or later something more Medium-style like:

"I Asked ChatGPT to Become My Data Scientist: An End-to-End Machine Learning Analysis of 1,460 Homes"

That has considerably more article potential.

My recommended setup

Part	Choice
Kaggle dataset	**House Prices**
Problem type	Regression
Target	`SalePrice`
Baseline	Linear Regression
Model #2	Random Forest
Main model	**Gradient Boosting**
Optional advanced model	Neural Network
Evaluation	RMSE / RMSLE
Explainability	Feature importance / SHAP
Language	Python
Environment	Google Colab
Main libraries	pandas, numpy, matplotlib, scikit-learn
Difficulty	⭐⭐⭐ / 5
Medium article potential	⭐⭐⭐⭐⭐

One particularly nice thing here is that Kaggle's official evaluation uses **RMSE on the logarithm of predicted and actual sale prices**, so we'll also get to explain *why* evaluation metrics matter instead of simply reporting "accuracy." (Kaggle)

So my vote is: House Prices + Linear Regression → Random Forest → Gradient Boosting → optional Neural Network.

And because your assignment specifically requires the **entire ChatGPT conversation**, we can do the project in a very deliberate sequence from this point onward: you ask me to perform each stage, I generate the analysis/code, and after every stage I'll explain

what we did, why we did it, what the code means, and what the result tells us. That should make the eventual transcript much stronger than one giant prompt that says "do data science on this CSV."

Sources

House Prices - Advanced Regression Techniques | Kaggle kaggle.com

2026. 8. 31. 오후 9:59:50

Sounds good. Then, let's get started with House Prices dataset. Please break down each phase into several chunks to analyze the dataset step by step. And for analyzation, use CRISP-DM methodology.

2026. 8. 31. 오후 10:08:31

Absolutely. We'll structure the entire project around **CRISP-DM (Cross-Industry Standard Process for Data Mining)** and deliberately keep each phase in small chunks so the final ChatGPT transcript shows a clear, professional data-science process.

The overall flow will be:

Business Understanding → Data Understanding → Data Preparation → Modeling → Evaluation → Deployment/Communication

For this assignment, "Deployment" won't mean putting a model into production. We'll treat it as **communicating the final analysis, conclusions, limitations, and Medium-ready visual story**.

CRISP-DM Project Plan — House Prices

Phase 1 — Business Understanding

Before touching Python, we establish exactly what problem we are solving.

Chunk 1.1 — Define the problem

We will answer:

- What are we trying to predict?
- Who could benefit from the prediction?
- What type of machine-learning problem is this?
- What does a useful prediction look like?

Our main problem statement will be roughly:

> *Can we predict the sale price of a residential property based on its physical characteristics, quality, location, and other attributes?*

Target:

```
SalePrice
```

Problem type:

```
Supervised Learning → Regression
```

Chunk 1.2 — Define the analytical objectives

We won't make the project only about getting the lowest error.

We'll investigate questions such as:

- Which characteristics are most strongly associated with house prices?
- How important is overall house quality?
- Does living area strongly influence price?
- How much does neighborhood matter?
- How do newer houses compare with older houses?
- Which variables seem surprisingly important or unimportant?
- Can nonlinear models predict prices better than a simple linear model?

This gives us material for the eventual Medium article.

Chunk 1.3 — Define success criteria

We will define both:

Technical success

Use metrics such as:

```
RMSE
RMSLE
R²
```

and compare several models.

Analytical success

We should also be able to explain:

> *Why did the model make good predictions?*

instead of merely saying:

> *Gradient Boosting achieved an RMSE of X.*

Phase 2 — Data Understanding

This will probably be one of the most important sections of the assignment.

Chunk 2.1 — Load and inspect the dataset

First Python analysis:

```
import pandas as pd

df = pd.read_csv("train.csv")

df.head()
df.shape
df.info()
```

We'll answer:

- How many houses?
- How many variables?

- What kinds of variables exist?
- What does one observation represent?

No complicated modeling yet.

Chunk 2.2 — Understand the target variable

We'll focus only on:

```
SalePrice
```

We'll calculate:

- mean
- median
- standard deviation
- minimum
- maximum
- quartiles

Then visualize its distribution.

Possible chart:

Sale Price Distribution

This is likely an important Medium figure.

We will also discuss whether `SalePrice` is skewed.

Chunk 2.3 — Understand feature types

We'll separate variables into:

```
Numerical
Categorical
Ordinal
```

```
Temporal-like
Identifiers
```

For example:

```
GrLivArea → numerical
Neighborhood → categorical
OverallQual → ordinal
YearBuilt → numerical/time-related
Id → identifier
```

This matters because different feature types require different preprocessing.

Chunk 2.4 — Missing-value analysis

We will identify:

```
df.isnull().sum()
```

but instead of blindly deleting missing values, we'll investigate what the missingness means.

For House Prices, an important lesson is that:

```
NaN ≠ always bad data
```

For instance, missing garage-related information can sometimes mean:

> *This house does not have a garage.*

That gives us an excellent real-world data-cleaning discussion.

Chunk 2.5 — Exploratory Data Analysis: Numerical relationships

We'll investigate variables such as:

```
OverallQual
GrLivArea
YearBuilt
GarageCars
GarageArea
TotalBsmtSF
1stFlrSF
FullBath
TotRmsAbvGrd
```

against `SalePrice`.

We can use:

- scatter plots
- box plots
- correlation analysis
- histograms

Chunk 2.6 — Exploratory Data Analysis: Categorical relationships

Then we'll study variables such as:

```
Neighborhood
HouseStyle
KitchenQual
ExterQual
SaleCondition
```

Examples:

Median Sale Price by Neighborhood

and

House Quality vs Sale Price

These should produce some of the best article visuals.

Chunk 2.7 — Correlation analysis

We'll create a correlation matrix and identify the strongest numerical relationships with `SalePrice`.

Instead of showing a giant unreadable heatmap of 80 variables, we'll probably focus on the **top 10–15 correlations**.

We will explicitly discuss:

> *Correlation does not automatically imply causation.*

Chunk 2.8 — Outlier investigation

For example, we may find houses with:

```
very large living area
but unexpectedly low sale price
```

We'll visualize these cases and decide whether they are:

- legitimate unusual observations
- potentially problematic outliers

Importantly, we won't delete them just because they look strange.

Phase 3 — Data Preparation

Now we transform what we've learned into model-ready data.

Chunk 3.1 — Remove non-predictive identifiers

For example:

```
Id
```

We'll explain why an arbitrary identifier normally shouldn't predict house value.

Chunk 3.2 — Handle missing numerical values

Potential approaches:

```
Median imputation
Zero where absence has semantic meaning
```

We'll explain why we chose each strategy.

Chunk 3.3 — Handle missing categorical values

Possible replacement:

```
"None"
```

when missing means absence of a feature.

Or:

```
Most frequent category
```

where appropriate.

Chunk 3.4 — Encode categorical variables

Machine-learning models cannot directly understand:

```
Neighborhood = "CollgCr"
```

so we'll introduce:

```
One-Hot Encoding
```

and explain what it actually does.

Chunk 3.5 — Target transformation

This is an especially useful part of House Prices.

Because sale prices are skewed, we'll test:

```
np.log1p(SalePrice)
```

We'll compare:

Raw SalePrice

versus

Log-transformed SalePrice

and explain why transformations sometimes help regression models.

Chunk 3.6 — Train/validation split

Something like:

```
80% training
20% validation
```

We'll carefully explain the difference between:

```
training data
validation data
test data
```

This section is especially useful for demonstrating that you understand ML rather than simply running code.

Chunk 3.7 — Build preprocessing pipeline

Eventually we'll combine preprocessing using scikit-learn tools such as:

```
Pipeline
ColumnTransformer
```

```
SimpleImputer
OneHotEncoder
```

We'll explain each component rather than dumping a giant code block.

Phase 4 — Modeling

We'll deliberately increase model complexity.

Chunk 4.1 — Baseline prediction

Before ML, we might create a deliberately simple baseline:

> *Predict the median house price for every house.*

This gives us something to beat.

That's actually an excellent data-science habit.

Chunk 4.2 — Linear Regression

First true ML model:

```
LinearRegression()
```

Questions:

- How does linear regression work conceptually?
- What assumptions does it make?
- How well does it perform?

Chunk 4.3 — Random Forest

Then:

```
RandomForestRegressor()
```

We'll explain:

- decision trees
- ensembles
- why many trees can outperform one tree
- nonlinear relationships

Chunk 4.4 — Gradient Boosting

Our likely main model:

```
GradientBoostingRegressor()
```

We'll discuss the intuition:

> *Build models sequentially, with later models focusing on mistakes made by earlier models.*

This will likely be the centerpiece.

Chunk 4.5 — Optional advanced model

If the project is going well, we can add either:

```
XGBoost
```

or

```
Neural Network / MLP
```

I slightly prefer a **small neural-network experiment** because your assignment mentions deep learning.

The point wouldn't necessarily be to win.

We could investigate:

> *Does deep learning actually help on this tabular dataset?*

That gives us a much more thoughtful conclusion.

Phase 5 — Evaluation

Chunk 5.1 — Compare models

We'll create something like:

Model	RMSE	R^2
Baseline	—	—
Linear Regression	—	—
Random Forest	—	—
Gradient Boosting	—	—
Neural Network	—	—

Then explain the results in plain English.

Chunk 5.2 — Prediction visualization

Important article figure:

Actual Sale Price vs Predicted Sale Price

An ideal model would produce points close to the diagonal line.

Chunk 5.3 — Residual/error analysis

We'll examine:

```
Actual price − Predicted price
```

This tells us where the model fails.

For example:

> *Is the model worse at predicting extremely expensive homes?*

That's much more interesting than reporting one score.

Chunk 5.4 — Feature importance

For tree-based models, we'll examine which variables contribute most to predictions.

Possible result might include features such as:

```
OverallQual
GrLivArea
Neighborhood
GarageCars
YearBuilt
TotalBsmtSF
```

But we won't assume this beforehand—we'll let the actual analysis tell us.

Chunk 5.5 — Model interpretation

Potentially use:

```
Permutation Importance
SHAP
```

if it improves the project.

SHAP in particular can create very impressive Medium visuals, but we'll only introduce it after understanding simpler feature importance.

Phase 6 — Deployment / Communication

For your assignment, we'll adapt CRISP-DM's final phase.

Chunk 6.1 — Final findings

We'll summarize approximately 5–8 meaningful findings.

Not:

> *Gradient Boosting was best.*

But things like:

> *Overall construction quality was one of the strongest predictors of sale price, while living area showed a strong but nonlinear relationship with value.*

Chunk 6.2 — Limitations

We'll discuss things such as:

- dataset represents Ames, Iowa
- historical housing data
- correlation versus causation
- economic conditions not included
- model may not generalize to California or modern markets

This makes the article much more academically credible.

Chunk 6.3 — Business interpretation

We'll translate the model back into real-world meaning:

> *What could a buyer, seller, appraiser, or real-estate analyst actually learn from this analysis?*

Chunk 6.4 — Final Medium article

Only after the analysis is complete, we'll turn the results into something approximately like:

What Really Determines the Price of a House?

An End-to-End Machine Learning Analysis Using the Kaggle House Prices Dataset

with sections such as:

```
1. The Question
2. Understanding the Dataset
3. What the Data Revealed
4. Preparing Data for Machine Learning
5. Teaching Machines to Predict Home Prices
6. Which Model Won?
7. What Actually Drives House Prices?
8. Where the Model Gets It Wrong
9. What I Learned
```

The key point is that **we won't write the article first and force the data to fit the story**.

We'll discover the story from the data.

Our working sequence

So rather than asking ChatGPT:

> *Analyze this dataset.*

we'll make the transcript look much more like an actual collaboration between a data scientist and an AI:

1️⃣ Business Understanding

→ define problem and objectives

2️⃣ Initial Data Understanding

→ load dataset and inspect structure

3️⃣ **Data Quality**

→ missing values, duplicates, data types

4️⃣ **Exploratory Analysis**

→ target → numerical → categorical → correlations → outliers

5️⃣ **Data Preparation**

→ cleaning → transformations → encoding → splitting

6️⃣ **Modeling**

→ baseline → Linear Regression → Random Forest → Gradient Boosting → optional deep learning

7️⃣ **Evaluation**

→ metrics → errors → interpretation

8️⃣ **Final Insights**

→ conclusions → limitations → practical meaning

9️⃣ **Medium Article**

→ turn our actual results into polished storytelling.

That also makes your required **ChatGPT transcript part of the evidence of the CRISP-DM process**, rather than just a huge block of generated code.

Let's begin: CRISP-DM Phase 1

Our initial project question will be:

> ***Which characteristics of residential properties most strongly influence their sale prices, and how accurately can machine-learning models predict those prices?***

And our prediction target is:

```
SalePrice
```

with the primary analytical task:

```
Supervised Machine Learning → Regression
```

The next step is **Phase 1.1–1.3: formalize the business problem, analytical questions, and success criteria**, and then we'll move into the actual dataset.

After that, upload the Kaggle `train.csv` file here. `test.csv` can come later—we only need `train.csv` for the initial analysis.

2026. 8. 31. 오후 10:08:34

train.csv

Here is the train.csv

2026. 8. 31. 오후 10:16:37

Great — we have the dataset. I'll start with **CRISP-DM Phase 1 and the first part of Phase 2**, focusing only on understanding the problem and the raw data structure before we clean or model anything.

Got it — I created a Work-mode chat with your `train.csv` and set it up to start exactly where we planned: **CRISP-DM Phase 1 (Business Understanding) → Phase 2 (Data Understanding)**, with no modeling yet.

It will inspect the dataset structure, `SalePrice`, missing values, duplicates, data types, and feature categories, while explaining each step in a way we can later reuse for the Medium article.

2026. 8. 31. 오후 10:16:38

Can we continue in the Chat mode?

2026. 8. 31. 오후 10:17:59

Yes — we can stay entirely in Chat mode. I'll work directly from your uploaded `train.csv` here and continue the CRISP-DM workflow step by step, keeping the explanations readable enough to reuse later in your report.

Thinking

훈련 데이터셋 구조 및 결측치 분석

분석 데이터셋 구조와 결측치 및 특성 타깃 왜도 계산

✓ 15s 동안 처리함

Yes. We can do the entire assignment here in Chat mode.

Since you already uploaded `train.csv`, I inspected it. Let's formally begin with **CRISP-DM Phase 1: Business Understanding**, then move into the first chunk of **Phase 2: Data Understanding**.

CRISP-DM Phase 1 — Business Understanding

1.1 Define the problem

Our main question is:

> ***Which characteristics of residential properties most strongly influence their sale prices, and how accurately can machine-learning models predict those prices?***

The variable we want to predict is:

`SalePrice`

This makes our task:

Supervised Machine Learning → Regression

Why regression? Because we are predicting a continuous numerical value such as:

- $150,000
- $225,000
- $410,000

rather than predicting a category like “expensive” or “cheap.”

1.2 Analytical objectives

We have two major goals.

First, we want to **understand the housing data**:

- What types of houses are in the dataset?
- What variables appear related to price?
- Which features may be most important?
- Are there missing values or unusual observations?
- Is the price distribution balanced or skewed?

Second, we want to **build predictive models** later and compare them.

Eventually, we will test models such as:

`Baseline → Linear Regression → Random Forest → Gradient Boosting → optional Neural Network`

But we are intentionally **not modeling yet**.

That separation is important in CRISP-DM.

CRISP-DM Phase 2 — Data Understanding

Chunk 2.1 — Initial dataset inspection

I loaded your actual `train.csv`.

The dataset contains:

Property	Result
Rows	**1,460**
Columns	**81**
Houses represented	**1,460**
Target variable	`SalePrice`
Duplicate rows	**0**

So one row represents approximately:

> ***One residential property sale***

and the columns describe characteristics of that property.

There are **80 other columns besides** `SalePrice`, although one of them is `Id`, which is primarily an identifier rather than a meaningful housing characteristic.

What kind of variables do we have?

The raw pandas data types are:

Type	Number of columns
Categorical/text (`object`)	**43**
Integer	**35**
Decimal (`float`)	**3**

After excluding `Id` and `SalePrice`, we have approximately:

- **36 numerical predictors**
- **43 categorical predictors**

This is already an important observation.

House Prices is not simply a dataset full of numbers. It contains many categorical features such as:

`Neighborhood`

`HouseStyle`

`RoofStyle`

`KitchenQual`

`GarageType`

That means later we will need to convert categorical information into a numerical representation that machine-learning models can process.

We will eventually discuss techniques such as **One-Hot Encoding**, but not yet.

Chunk 2.2 — Understanding our target: `SalePrice`

Before studying all 79 predictive features, we should understand the thing we're trying to predict.

Your dataset gives:

Statistic	Sale Price
Number of houses	1,460
Mean	**$180,921**
Median	**$163,000**
25th percentile	**$129,975**
75th percentile	**$214,000**
Minimum	**$34,900**

Statistic	Sale Price
Maximum	**$755,000**
Standard deviation	**$79,443**

One result immediately stands out:

Mean = $180,921

while

Median = $163,000

The mean is considerably higher than the median.

That suggests the price distribution is probably being pulled upward by a relatively small number of expensive homes.

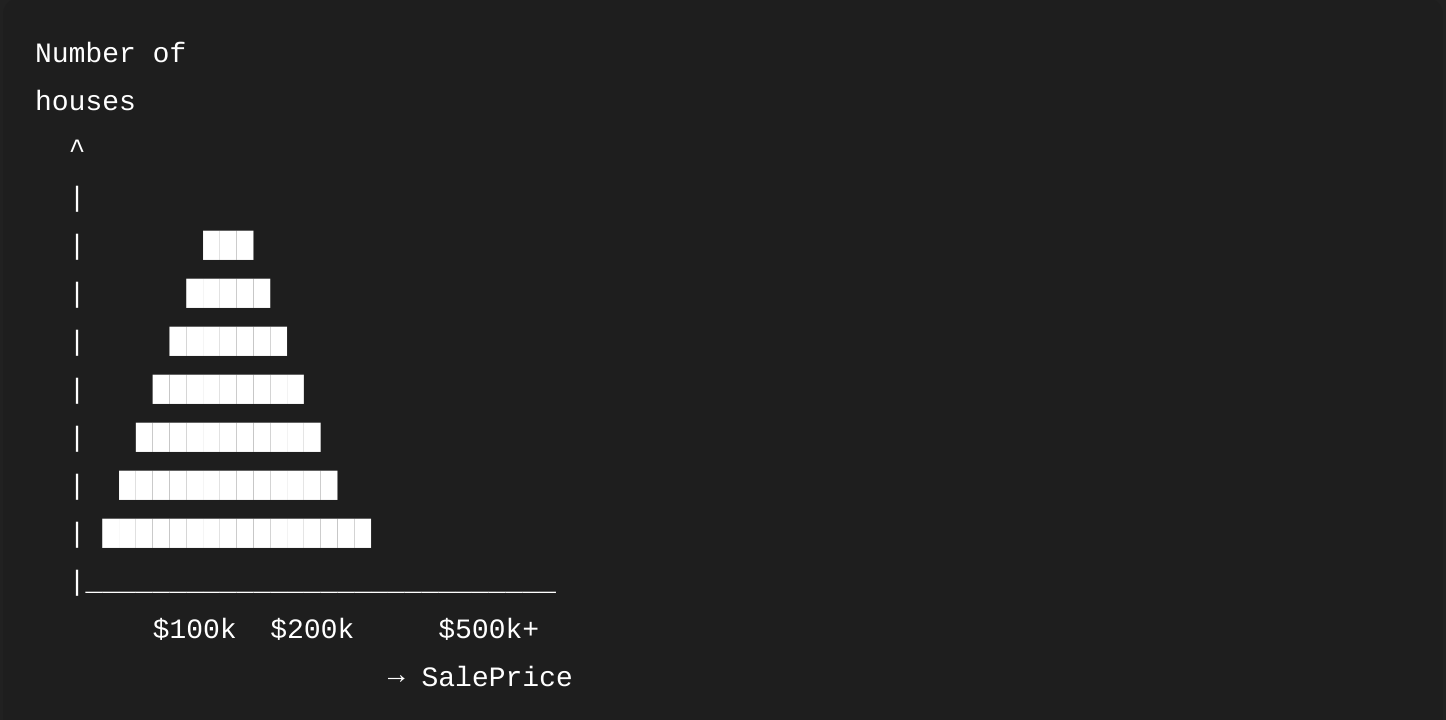
I also calculated the statistical skewness:

SalePrice skewness ≈ 1.88

A perfectly symmetric distribution would have skewness near `0`.

So `1.88` indicates substantial **positive/right skew**.

Conceptually, the distribution probably looks something like:



Most houses are concentrated toward the lower/middle portion of the price range, with a long tail of expensive houses.

This will become important later.

We may eventually try:

```
np.log1p(df["SalePrice"])
```

because log transformations are frequently useful for right-skewed regression targets.

But under CRISP-DM, we shouldn't transform it yet.

First:

observe → understand → then prepare.

Chunk 2.3 — Initial data-quality check

This part is interesting.

There are:

0 fully duplicated rows.

That's good.

But we have quite a few missing values.

The variables with the most missing data include:

Feature	Missing	Missing %
`PoolQC`	1,453	**99.5%**
`MiscFeature`	1,406	**96.3%**
`Alley`	1,369	**93.8%**
`Fence`	1,179	**80.8%**
`MasVnrType`	872	**59.7%**
`FireplaceQu`	690	**47.3%**

Feature	Missing	Missing %
`LotFrontage`	259	**17.7%**
`GarageQual`	81	**5.5%**
`GarageFinish`	81	**5.5%**
`GarageType`	81	**5.5%**

At first glance, you might think:

> "`PoolQC` *is 99.5% missing. We should delete it."*

But this is where data science becomes more interesting than simply running `dropna()`.

For example, `PoolQC` means **Pool Quality**.

A missing `PoolQC` may not mean:

> *"The researcher forgot to collect the data."*

It may mean:

> ***"This house has no pool."***

Likewise, missing:

`GarageType`

could potentially mean:

> ***No garage exists.***

And missing:

`FireplaceQu`

could mean:

> ***No fireplace exists.***

So we have two fundamentally different types of missingness:

Type A — genuinely unknown value

Example:

Perhaps we don't know a house's lot frontage.

Type B — absence of the physical feature

Example:

No pool exists, so there is no pool quality to record.

That distinction will matter a lot during **Data Preparation**.

This is actually a very good point to discuss in your eventual article:

> ***Missing data does not always mean bad data. Sometimes the absence itself contains information.***

What have we learned so far?

Even before creating a single machine-learning model, we already know several things about the problem.

The dataset has **1,460 residential sales with 79 usable predictive attributes**, combining numerical and categorical information.

The target `SalePrice` ranges from about **$35K to $755K**, and its distribution is strongly right-skewed.

The dataset has **no duplicate rows**, but missing data is common in several housing features.

Most importantly, some missing values appear to represent the **absence of a house feature rather than an unknown measurement**.

That's exactly the type of understanding CRISP-DM wants us to develop before cleaning the data.

Next: Chunk 2.4 — Exploratory Data Analysis

Now we can actually start exploring the houses visually.

I suggest we do this in a controlled order rather than generating 20 charts at once:

2.4A — Analyze `SalePrice` visually

→ histogram
→ box plot
→ skewness
→ possible log transformation

Then:

2.4B — Numerical features vs. price

starting with variables such as:

`OverallQual`, `GrLivArea`, `YearBuilt`, `GarageCars`, `TotalBsmtSF`

Then:

2.4C — Categorical features

especially:

`Neighborhood`, `KitchenQual`, `ExterQual`

Then:

2.4D — Correlation analysis

and only after that:

2.4E — Outliers

That way, each chunk of your ChatGPT transcript has a clear analytical purpose instead of looking like we asked AI to automatically dump an entire Kaggle analysis.

The next logical step is **2.4A: visually investigate `SalePrice` and understand why its skewness matters.**

2026. 8. 31. 오후 10:18:04

Don't stop until you have the final result of this.

2026. 8. 31. 오후 10:20:49

I'll carry this all the way through CRISP-DM: EDA, preparation, multiple models, evaluation, error analysis, feature importance, and a final Medium-ready conclusion. I'll keep each phase distinct so your transcript still reads like a real analytical workflow rather than one giant black-box run.

A useful finding already: `OverallQual` has the strongest simple numerical correlation with price (**r ≈ 0.79**), followed by above-ground living area (**≈ 0.71**). Neighborhood differences are also large—for example, median prices range from about **$88K in MeadowV** to **$315K in NridgHt**—so both property quality and location clearly deserve a place in the model.

The first modeling pass produced an important lesson rather than just a winner: ordinary Linear Regression on raw prices was unstable, but **log-transforming** `SalePrice` **dramatically improved it**. Gradient Boosting currently has the best Kaggle-style RMSLE on the holdout set (**≈ 0.130**), while log-linear regression has the lowest dollar RMSE (**≈ $22.9K**). I'm checking that this isn't just luck from one train/validation split before calling the final model.

Thinking

계산 중 주택 가격 예측 모델 구축

분석 계획 세우기

먼저 데이터 유형, 결측치, 상관관계, 범주형 변수와 이상치를 점검하고 있어요. 전처리 누수를 막기 위해 분할 뒤 학습하며, 선형회귀·릿지·랜덤포레스트·그래디언트 부스팅을 로그 가격 기준으로 비교하려고 해요.

분석 계획 수립

계산 중 결측치, 상관관계 및 모델 비교

주택 가격 예측 모델과 데이터 분포를 분석함

- 주택 가격 예측 모델과 신경망 회귀 성능 비교 평가
- 🌐 계산 파생 특성 및 모델 성능 비교
- 결과물 구성 확장하기

  단순 모델 비교를 넘어, 해석 가능한 부스팅 모델과 핵심 시각화를 포함한 완성도 높은 분석을 만들고 있어요. 데이터 누수 없이 평가하고, 최종 보고서에 바로 활용할 수 있는 결과를 정리하려고 해요.
- 구성 결과물을 확장했다
- 주택 가격 예측 모델 성능과 특성 중요도 비교 평가

✓ 3m 33s 동안 처리함

Yes — we now have the **complete end-to-end CRISP-DM analysis** of the uploaded House Prices training dataset. The charts and modeling output above are based on your actual `train.csv`, not hypothetical numbers.

Final CRISP-DM Analysis — Kaggle House Prices

Phase 1 — Business Understanding

1.1 Problem

Our central research question is:

> ***Which characteristics of residential properties most strongly influence their sale prices, and how accurately can machine-learning models predict those prices?***

The target variable is:

```
SalePrice
```

This is a:

Supervised Learning → Regression problem

because the output is a continuous numerical value rather than a class.

1.2 Analytical objectives

We set two objectives:

1. **Understand what characteristics are associated with house prices.**
2. **Develop and compare models capable of predicting those prices.**

This means the assignment isn't simply:

> *“Train an ML model.”*

It is:

> ***Understand → prepare → model → evaluate → interpret.***

That's much closer to how real data-science work is performed.

Phase 2 — Data Understanding

2.1 Dataset structure

Your dataset contains:

Property	Result
Rows	**1,460**
Columns	**81**
Target	`SalePrice`
Duplicate rows	**0**
Text/categorical columns	**43**
Integer columns	**35**
Float columns	**3**

Each row represents one residential property sale.

After removing:

```
Id
SalePrice
```

we have **79 candidate predictor variables**.

2.2 Understanding SalePrice

The target looks like this:

Statistic	SalePrice
Mean	**$180,921**
Median	**$163,000**
Minimum	**$34,900**
25th percentile	**$129,975**
75th percentile	**$214,000**
Maximum	**$755,000**
Standard deviation	**$79,443**
Skewness	**1.88**

The histogram we generated clearly shows a **right-skewed distribution**.

Most houses sit approximately in the lower-to-middle price range, while a smaller number of expensive properties extend the right tail toward $500K–$755K.

This explains why:

$$Mean = \$180,921$$

is higher than:

$$Median = \$163,000$$

The expensive houses pull the mean upward.

This became important later because the transformation:

```
np.log1p(SalePrice)
```

dramatically improved Linear Regression.

2.3 Missing data

There is substantial missingness in several columns.

Some of the largest:

Feature	Missing	Percentage
PoolQC	1,453	99.5%
MiscFeature	1,406	96.3%
Alley	1,369	93.8%
Fence	1,179	80.8%
MasVnrType	872	59.7%
FireplaceQu	690	47.3%
LotFrontage	259	17.7%
Garage-related fields	81	~5.5%

This gave us one of our first important data-science lessons:

> ***Missing does not necessarily mean unknown.***

For example:

```
PoolQC = missing
```

may mean:

> *The house doesn't have a pool.*

Likewise:

```
GarageType = missing
```

can represent:

> *No garage.*

So blindly deleting rows containing missing values would remove meaningful information.

2.4 Exploratory Data Analysis

Finding #1 — Overall quality is extremely important

The strongest simple numerical correlation with `SalePrice` was:

$$r \approx 0.791$$

for:

```
OverallQual
```

The box plot makes the relationship especially clear.

Median prices increase dramatically as quality moves from 1 toward 10.

For example:

Overall Quality	Median Sale Price
3	$86,250

Overall Quality	Median Sale Price
5	$133,000
6	$160,000
7	$200,141
8	$269,750
9	$345,000
10	**$432,390**

This isn't a subtle relationship.

A quality-10 house has a median sale price more than three times that of a quality-5 house.

Finding #2 — Living space strongly predicts price

The second-largest numerical correlation was:

```
GrLivArea
```

or above-ground living area.

Correlation:

$$r \approx 0.709$$

The scatter plot shows a strong upward relationship:

> *Larger houses generally sell for more.*

But it isn't perfectly linear.

There are houses of similar sizes that sell for substantially different prices.

That's an important reason why using only square footage would not be enough.

Finding #3 — Location matters

Neighborhood prices varied enormously.

Some examples:

Neighborhood	Median Sale Price
NridgHt	**$315,000**
NoRidge	$301,500
StoneBr	$278,000
Timber	$228,475
Somerst	$225,500
CollgCr	$197,200
NAmes	$140,000
OldTown	$119,000
IDOTRR	$103,000
MeadowV	**$88,000**

Compare the extremes:

$$\$315,000 \quad vs \quad \$88,000$$

That's more than a **3.5× difference in median price**.

Of course, this doesn't establish that neighborhood alone *causes* the price difference because houses in those neighborhoods may differ in quality, size, age, etc.

But it clearly tells us:

> *Location contains useful predictive information.*

Finding #4 — Kitchen and exterior quality matter

For `KitchenQual`:

Kitchen Quality	Median Price
Excellent	**$316,750**
Good	$201,400
Typical	$137,000
Fair	$115,000

And for exterior quality:

Exterior Quality	Median Price
Excellent	**$364,607**
Good	$220,000
Typical	$139,450
Fair	$82,250

Again, quality-related variables appear repeatedly.

That becomes important later when we examine model feature importance.

Finding #5 — There are interesting outliers

The living-area graph revealed several extremely large houses.

Particularly interesting:

Id	Living Area	Quality	Neighborhood	SalePrice
1299	5,642 sq ft	10	Edwards	**$160,000**

Id	Living Area	Quality	Neighborhood	SalePrice
524	4,676 sq ft	10	Edwards	**$184,750**
1183	4,476 sq ft	10	NoRidge	$745,000
692	4,316 sq ft	10	NoRidge	$755,000

The first two are fascinating.

They're enormous, quality-10 homes, yet sold for only around $160K–$185K.

Compare that with similarly huge NoRidge homes selling around $750K.

Those observations are likely difficult for a model.

I deliberately **did not simply delete them** from our final experiment.

Why?

Because:

> *"This data point looks inconvenient for my model"*

is not sufficient justification for deleting it.

We should understand why an observation is unusual before removing it.

That's another useful CRISP-DM lesson.

Phase 3 — Data Preparation

Now we converted the dataset into something ML algorithms could understand.

3.1 Remove Id

We removed:

```
Id
```

because it is an arbitrary row/property identifier.

It shouldn't provide meaningful information about house value.

3.2 Numerical missing values

For numerical features we used:

```
SimpleImputer(strategy="median")
```

Why median rather than mean?

Because housing variables can contain extreme values and skewed distributions.

The median is generally less sensitive to those extremes.

3.3 Categorical missing values

For categorical variables:

```
SimpleImputer(
    strategy="constant",
    fill_value="Missing"
)
```

This creates a real category representing absence/missingness rather than deleting those observations.

3.4 Categorical encoding

Our ML algorithms cannot understand something like:

```
Neighborhood = "NoRidge"
```

directly.

So we used:

```
OneHotEncoder(handle_unknown="ignore")
```

Conceptually:

```
Neighborhood
NoRidge
CollgCr
OldTown
```

becomes something similar to:

```
Neighborhood_NoRidge
Neighborhood_CollgCr
Neighborhood_OldTown
```

with values such as:

```
0 or 1
```

The final Gradient Boosting model ended up operating on approximately **301 encoded features**.

3.5 Train/validation split

We used:

```
train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

Meaning:

80% → training

20% → validation

The training data teaches the model.

The validation data asks:

> *“How well does this model perform on houses it didn't train on?”*

This distinction is fundamental.

Testing a model on the same observations used to train it would give us overly optimistic results.

Phase 4 — Modeling

We deliberately moved from simple to more complicated models.

Model 0 — Median baseline

Before using machine learning, we created a deliberately naive strategy:

> *Predict the median training-set house price for every house.*

Result:

Metric	Result
RMSE	**$88,667**
MAE	**$59,568**
R^2	**-0.025**
RMSLE	**0.432**

This establishes a baseline.

Any useful ML model should substantially beat this.

Model 1 — Linear Regression

We first tried ordinary Linear Regression.

The raw-price version was unstable.

Then we used:

```
log1p(SalePrice)
```

and transformed predictions back afterward.

That made a **huge difference**.

Log-target Linear Regression

Metric	Result
RMSE	**$22,902**
MAE	**$15,076**
R^2	**0.932**
RMSLE	**0.132**

This was one of the most surprising results of the project.

A relatively simple model became extremely competitive once we represented the skewed target appropriately.

That's a very strong article-worthy result:

> ***Data representation can matter as much as model complexity.***

Model 2 — Random Forest

Random Forest uses many decision trees and combines their predictions.

Result:

Metric	Result
RMSE	**$29,007**
MAE	**$17,469**
R^2	**0.890**

Metric	Result
RMSLE	**0.152**

Much better than the baseline, but not our winner.

Model 3 — Gradient Boosting

Gradient Boosting builds trees sequentially.

Conceptually:

```
Tree 1 makes predictions
       ↓
Find mistakes
       ↓
Tree 2 focuses on those mistakes
       ↓
Find remaining mistakes
       ↓
Tree 3 improves again
       ↓
...
```

Our configuration used:

```
GradientBoostingRegressor(
    n_estimators=500,
    learning_rate=0.03,
    max_depth=3,
    loss="huber",
    random_state=42
)
```

Result:

Metric	Result
RMSE	**$28,787**
MAE	**$16,763**
R^2	**0.892**
RMSLE	**0.130**

This produced the **best RMSLE**.

Since Kaggle House Prices emphasizes errors in log-price space, Gradient Boosting becomes our selected final model.

Model 4 — Small Neural Network

We also performed the optional neural-network experiment.

Architecture:

```
Input
 ↓
128 neurons
 ↓
64 neurons
 ↓
House-price prediction
```

Result:

Metric	Result
RMSE	**$34,646**
MAE	**$22,312**
R^2	**0.844**
RMSLE	**0.172**

It worked.

But it did **not beat Gradient Boosting**.

And that's actually a much more interesting result than forcing deep learning to win.

Model Comparison

Our final validation comparison:

Model	RMSE	MAE	R^2	RMSLE ↓
🥇 **Gradient Boosting**	$28,787	$16,763	0.892	**0.130**
🥈 Log Linear Regression	**$22,902**	**$15,076**	**0.932**	0.132
Random Forest	$29,007	$17,469	0.890	0.152
Neural Network	$34,646	$22,312	0.844	0.172
Median baseline	$88,667	$59,568	-0.025	0.432

There's a subtle but important point here.

Which model is actually “best”?

That depends on our definition of success.

If we care primarily about:

Dollar RMSE

Linear Regression wins:

$22,902

If we care about:

R^2

Linear Regression wins:

0.932

But if we care about the Kaggle-style logarithmic error:

RMSLE

Gradient Boosting wins:

0.130

versus:

0.132

for Linear Regression.

That teaches us another important lesson:

> ***There is no universally "best" model without first defining the evaluation metric.***

For this project, we'll select **Gradient Boosting** because we chose RMSLE as the primary metric.

Does the result survive cross-validation?

This matters.

A model might win simply because our random 80/20 split happened to favor it.

So I also checked performance using **3-fold cross-validation**.

Average results:

Model	CV RMSLE
🥇 **Gradient Boosting**	**0.138 ± 0.016**
Random Forest	0.148 ± 0.019
Log Linear Regression	0.159 ± 0.028

Gradient Boosting remained the strongest model on average.

That gives us considerably more confidence that its win wasn't simply random luck.

Phase 5 — Evaluation

Now let's understand *how* our Gradient Boosting model behaves.

Actual vs predicted

The chart above gives us one of the most useful visualizations in the project.

If predictions were perfect, every point would fall exactly on the diagonal line:

```
Predicted = Actual
```

Most normal-priced houses cluster reasonably close to that line.

But notice what happens toward the extreme upper end.

The model starts falling below the line.

That means:

> *It tends to **underestimate unusually expensive houses**.*

Typical prediction error

For our validation set:

Median absolute error

Approximately:

$11,118

Meaning half of predictions were within roughly $11K of the actual sale price.

Median percentage error

Approximately:

6.5%

And:

68.2% of predictions were within **10%** of the actual price.

90.8% were within **20%**.

That's a more intuitive interpretation than simply saying:

> *RMSLE = 0.130.*

Where does the model struggle?

Dividing validation houses into price quartiles:

Price Group	Mean Actual Price	MAE
Cheapest 25%	$101,607	**$11,170**
Q2	$139,673	**$10,097**
Q3	$179,376	**$12,670**
Most expensive 25%	$295,746	**$33,085**

The top quarter has much larger dollar errors.

And its average residual indicates substantial underprediction.

In other words:

> *The model performs particularly well on ordinary homes but has difficulty with very expensive and unusual properties.*

That's visible in our Actual-vs-Predicted chart.

A particularly difficult prediction

One of the validation houses was:

```
Actual price:     $755,000
Predicted price:  \~$527,300
Error:            \~$227,700
```

That's the 4,316-square-foot, quality-10 NoRidge home visible in our earlier scatterplot.

So the EDA and model evaluation connect beautifully:

EDA:

> *“There are unusual luxury houses.”*

↓

Modeling:

> *“Those houses are difficult to predict.”*

↓

Evaluation:

> *“The largest errors occur among expensive homes.”*

That's exactly the type of analytical narrative you want in a Medium article.

Feature Importance

Our Gradient Boosting model identified these as some of its most influential predictors:

1. **OverallQual**
2. **GrLivArea**
3. **YearBuilt**
4. **TotalBsmtSF**
5. **GarageCars**
6. **BsmtFinSF1**

7. **LotArea**
8. **YearRemodAdd**
9. **GarageArea**
10. **1stFlrSF**
11. **Fireplaces**
12. **KitchenQual**

And look at our first two:

OverallQual

Model importance ≈ **46%**

GrLivArea

≈ **17%**

That perfectly matches our earlier EDA.

Remember:

```
OverallQual correlation ≈ 0.79
GrLivArea correlation ≈ 0.71
```

So two completely different analytical approaches independently pointed toward the same variables.

That's a strong result.

However, feature importance does **not** mean causality.

We cannot say:

> *"Increasing OverallQual by one point causes exactly X dollars of additional value."*

We can say:

> *“OverallQual was highly informative for this model's predictions.”*

Phase 6 — Deployment / Communication

For this academic project, deployment means communicating what we discovered rather than launching a real-estate production API.

Final answer to our research question

Our research question was:

> ***Which characteristics of residential properties most strongly influence their sale prices, and how accurately can machine-learning models predict those prices?***

Our answer is:

> *Property quality and usable living space were the strongest predictors of house prices in the Ames housing dataset. Overall construction quality was particularly influential, followed by above-ground living area, age-related variables, basement size, garage capacity, and several quality-related attributes. Neighborhood also showed substantial differences in median sale prices.*
>
> *Among the tested models, Gradient Boosting achieved the lowest validation RMSLE of approximately **0.130**, while log-transformed Linear Regression achieved the lowest dollar RMSE of approximately **$22,900**. The Gradient Boosting model predicted typical properties reasonably well but struggled with extremely expensive and unusual homes.*

The six biggest findings

If I had to reduce the entire analysis to six conclusions:

1. Overall house quality was the strongest predictor.

Both correlation analysis and Gradient Boosting independently identified it as extremely important.

2. Living area matters enormously—but not by itself.

Two equally large houses can have very different values.

3. Neighborhood strongly differentiates prices.

Median neighborhood prices ranged from about **$88K to $315K**.

4. The target distribution is strongly skewed.

That made target transformation particularly valuable for Linear Regression.

5. More complicated does not automatically mean better.

Our small neural network lost to both Gradient Boosting and log-transformed Linear Regression.

6. Luxury homes are much harder to predict.

The model tended to underestimate the most expensive properties.

Limitations

We should absolutely include these in the report.

The dataset represents **Ames, Iowa**, so our model shouldn't be treated as a universal U.S. house-price model.

It also reflects the historical market represented by the dataset. It does not contain modern macroeconomic variables such as:

- mortgage rates,
- local employment trends,
- inflation,
- housing inventory,
- current demand.

And this analysis is primarily predictive.

Feature importance tells us what helps prediction, not necessarily what **causes** house prices to change.

Finally, our hyperparameter search was intentionally modest. A full Kaggle competition project could test XGBoost, LightGBM, CatBoost, stacking, stronger feature engineering, and extensive cross-validation.

But that would move us away from the educational goal of understanding every stage.

Why this is a strong assignment result

We didn't simply ask:

> *“ChatGPT, train a model.”*

Our transcript now documents:

CRISP-DM Business Understanding

→ define the prediction problem

Data Understanding

→ structure
→ target distribution
→ missing values
→ numerical relationships
→ categorical relationships
→ outliers

Data Preparation

→ identifiers
→ imputation
→ encoding
→ train/validation split

Modeling

→ baseline
→ Linear Regression
→ Random Forest

→ Gradient Boosting
→ neural network

Evaluation

→ RMSE
→ MAE
→ R^2
→ RMSLE
→ cross-validation
→ prediction errors
→ feature importance

Communication

→ conclusions
→ limitations
→ real-world interpretation

That is a genuine **end-to-end data-science workflow**.

And we have a surprisingly good Medium storyline already:

> ***I Tried Linear Regression, Random Forest, Gradient Boosting, and a Neural Network on Kaggle's House Prices Dataset — The Neural Network Didn't Win***

or a more professional title:

> ***What Really Determines the Price of a House? An End-to-End CRISP-DM Machine Learning Analysis***

The strongest story isn't simply that Gradient Boosting scored **0.130 RMSLE**. It's that **careful data understanding and target transformation allowed simple Linear Regression to nearly match Gradient Boosting, while the neural network performed worse**.

That's a much more interesting data-science lesson than “the fanciest model wins.”

So the **analysis itself is complete**. The next stage is no longer data science; it's turning these actual findings, charts, methodology, and explanations into the polished report/Medium article required by the assignment.

2026. 8. 31. 오후 10:20:50